

SIT 218/738: Secure coding

Pass task 5.1P: SQL injection

Objective

SQL injection attack prevention requires both input validations and safe SQL statements. In this task you will address SQL injection attack using prepared statements that bind the user input after the query is prepared using the **PreparedStatement()** method. Additionally, Hibernate ORM uses **PreparedStatement()** by default, so you will instead need to analyse safe SQL statements used in Hibernate ORM that will address the SQL injection attacks.

Note: Students enrolled in SIT738 need to complete additional mandatory Tasks as mentioned below.

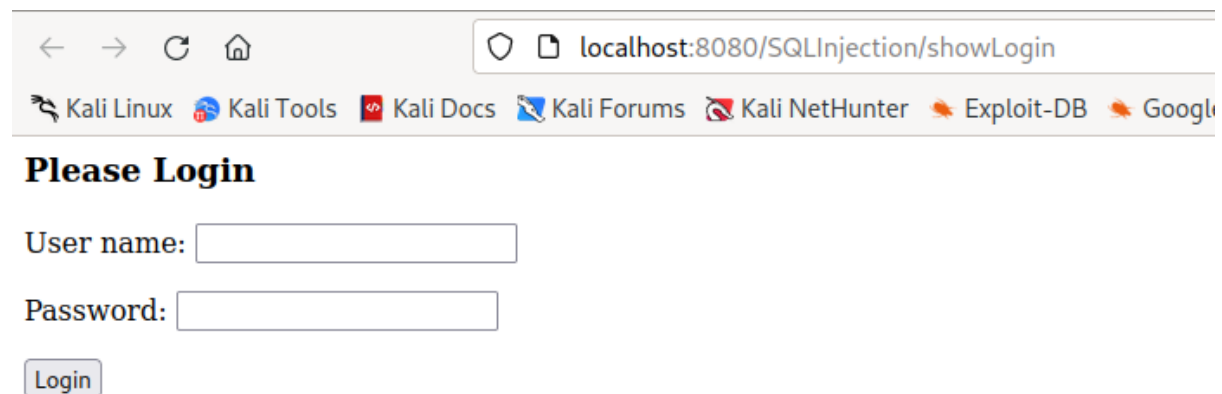
Part - 1:

In this part of this task, you will use the SQL injection technique to bypass the authentication mechanism used in web app. You will also identify the reasons for this attack to be successful.

Overview

Task-1: Complete the steps of all activities (Activity-1 to Activity -4) provided in “Ontrack Task5.1P Start Activity” on CloudDeakin.

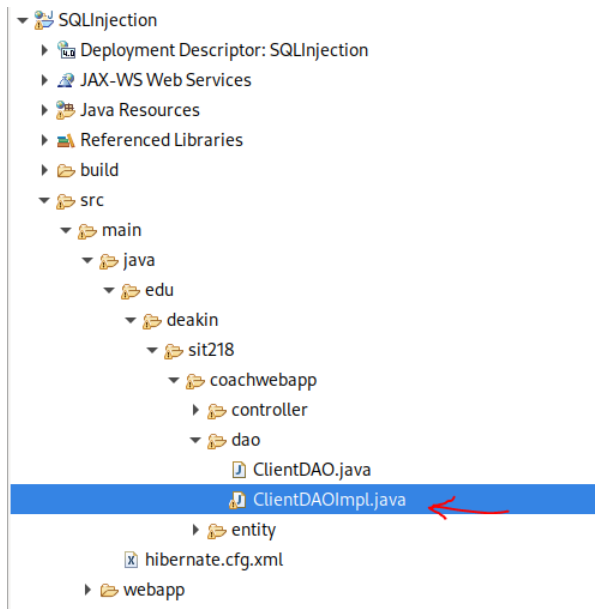
Task2: The SQLInjection webapp is vulnerable to SQL injection attacks. One of the users configured in the webapp has the username ‘Alice’. Craft a SQL injection code to bypass the authentication of the web app. You must continue from the steps given under the “**SQL injection test in webapp**” in the “Ontrack Task 5.1P Start activity” document and start the vulnerable webapp.



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/SQLInjection/showLogin'. The browser's toolbar includes icons for navigation and a search bar. Below the toolbar, there are several links: 'Kali Linux', 'Kali Tools', 'Kali Docs', 'Kali Forums', 'Kali NetHunter', 'Exploit-DB', and 'Google'. The main content area of the browser displays the text 'Please Login' in a bold, black font. Below this text, there are two input fields: 'User name:' followed by a text box, and 'Password:' followed by a text box. At the bottom of the form, there is a 'Login' button.

Once the above page loads, try to bypass the authentication.

Task3: Identify and highlight the part of vulnerable code in the functions **existsClient()** and **areCredentialsCorrect()**. The vulnerable code file is located in the ClientDAOImpl.java file as shown below:



Submission Requirements-Part 1:

Submit one PDF file containing the following information:

1. Provide screenshots and codes as necessary in your submission to demonstrate that you have completed Activity-1 to Activity -4. of Task-1
2. Provide SQL injection code which was used to bypass authentication using username 'Alice' in Task-2. Screenshot showing the injection code entered to the correct field and showing bypassing of the authentication in SQLInjection webapp in Task-2.
3. Screenshot of the line of code that is vulnerable and a brief explanation of the vulnerability in the code in Task 3.

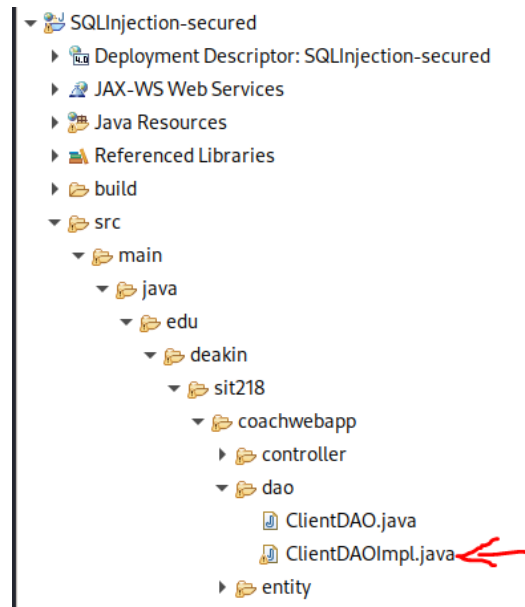
Part - 2:

For unit both SIT218 and SIT738 (Task 1 and Task 2):

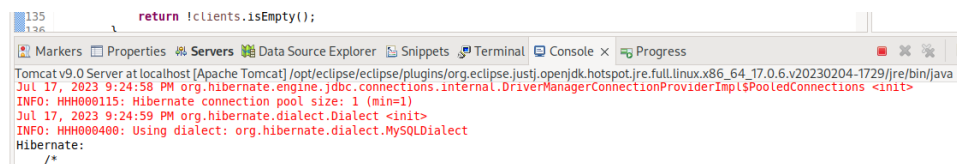
Task 1: In the SQLInjection web app used for Task 5.1P, re-write the vulnerable query in the `existsClient()` method of the `ClientDAOImpl.java` with a safe SQL query. Your safe query should use `PreparedStatement()` and `setString()` methods similar what we used in the `SQLInjectionTest.java` program discussed in the "Ontrack 5.1P Start Activity". (No need to run and demonstrate this code using screenshots)

Task 2: Hibernate ORM uses `PreparedStatement` by default to generate SQL queries. Additionally, it uses parameter binding to generate safe SQL queries. For this task you are required to:

- a. Access the **SQLInjection-secured** webapp into your Eclipse IDE as shown below figure.
- b. Highlight the secure SQL queries in the `existsClient` and `areCredentialsCorrect` functions in `ClientDAOImpl.java` file located in the `src` folder under `Dao`.



- c. Execute the application and try to bypass credentials for username 'Alice' using an SQL injection attack and record the response on web page and console log from Hibernate as shown below. You can view the exact query passed to the SQL server in the Eclipse Console.



For unit SIT738 only (mandatory):

Task 3: Rewrite the code of `existsClient(String username)` and `areCredentialsCorrect(String username, String password)` functions of `ClientDAOImpl.java` using JPA-style ordinal parameters to set the value of username and password instead of named parameter in the HQL query string. Then run your web application. Perform an SQL injection attack similar to Task-2 for user account 'Alice'.

[Hints: Query Parameters in JPA (objectdb.com), <https://www.objectdb.com/java/jpa/query/parameter>]

Submission Requirements—Part 2:

Submit one PDF file containing the following information:

4. Screenshots of the correct part of code using `PreparedStatement()` that demonstrates completion of Task-1 .
5. Submission requirement of Task-2
 - a. Identify and explain the parameter binding code used in Hibernate ORM in the `existsClient()` and `areCredentialsCorrect()` functions in `ClientDAOImpl.java` file located in the `src` folder under `Dao`
 - b. Screenshot of the application's response when targeted with SQL injection attack for user account 'Alice'

- c. Screenshot of the SQL queries generated by Hibernate in the Eclipse console
 - d. Explain the execution flow of the web app of Task-2 using Spring MVC framework and Hibernate ORM and how they are preventing SQL injection attacks. Provide a spring MVC diagram using the related components of the Task-2 webapp to demonstrate the execution flow and relationships of different components (views, controllers, model, databases). Your explanation of execution flow should use related codes from the webapp and reference to the different components of MVC framework such as views, front controller- Dispatch servlet, app controllers (loginControllers), RequestMappings used in the app, Persistent object used in the app, required configurations in hibernate.cfg.xml, spring-configuration.xml, web.xml, how error message is fed back to the user for input-errors in the login page.
6. Submission requirement of Task-3
- a. Copy of the updated code for functions **existsClient()** , **areCredentialsCorrect()** . Explain your code in the **existsClient()** and **areCredentialsCorrect()** functions in ClientDAOImpl.java file. Screenshot of the application's response when targeted with an SQL injection attack for user account 'Alice'.